

YouTube upload plan (draft for review)

Settings for every video (your defaults): made for kids **no** · altered/synthetic-content label **no** (AI help disclosed in the description instead) · category **Education** · privacy **private** (forced until the Google audit passes; you flip to public in YouTube Studio) · playlists created **private**.

Order: Foundations (14) first, then How LLMs Work (14), one series at a time, max 6 uploads a day, starting just after midnight Pacific. About 5 days in total.

Every description ends with the Manim credit and the AI-assistance disclosure. YouTube's API cannot post or pin comments; if you also want them as a pinned comment, paste the last two paragraphs of a description as a comment in Studio and pin it.

Playlist: How LLMs Work: Foundations

Short videos on the math and code behind large language models: vectors, dot products, matrices, nonlinearity, exponentials and logs, probability, softmax, statistics, waves, gradients, NumPy, neural networks, autograd and train/validation data. Each video points to the How LLMs Work episodes that use it.

1. Vectors: Lists of Numbers as Arrows | How LLMs Work: Foundations F01

VectorsVideo.mp4

The basic building block of every LLM.

This video has one big idea: a vector is a list of numbers that you can also picture as an arrow. You can add vectors, stretch them and measure their length, and the rules are the same whether a vector has 2 numbers or 768. Every token inside an LLM is represented by one.

Chapters

0:00 A vector is a list of numbers

0:27 Two numbers, one arrow

0:51 Adding vectors

1:02 Scaling: stretch, shrink, flip

1:13 Length, by Pythagoras · Directions that mean something

1:32 Vectors in NumPy

Foundations F01, a pre-series for How LLMs Work. Used in episodes 1, 3, 4 and 9.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

2. The Dot Product | How LLMs Work: Foundations F02

DotProductVideo.mp4

One number that says how much two vectors agree.

This video has one big idea: the dot product turns two vectors into one number that says how much they point the same way. The recipe is multiply matching numbers, then add. Divide by both lengths and you get cosine similarity, a pure measure of direction. LLMs use dot products almost everywhere.

Chapters

0:00 Multiply matching numbers, then add

0:37 Positive, zero, negative: agreement

0:50 The shadow and the angle

1:07 Cosine similarity

1:21 Dot products everywhere

1:33 The dot product in NumPy

Foundations F02, a pre-series for How LLMs Work. Used in episodes 3, 5, 6, 8 and 10.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

3. Matrices: Many Dot Products at Once | How LLMs Work: Foundations F03

MatricesVideo.mp4

The table of numbers behind every layer.

This video has one big idea: a matrix is a table of numbers that does many dot products at once. Matrix times vector dots every row with the vector; matrix times matrix dots every row with every column. That is how one layer of an LLM transforms every token in a single step.

Chapters

0:00 A matrix is a table of numbers

0:25 Matrix times vector

0:36 A matrix transforms space

0:47 The shapes must line up

0:58 Matrix times matrix: every token at once

1:24 The transpose

1:36 Matrices in NumPy

Foundations F03, a pre-series for How LLMs Work. Used in episodes 5, 6, 7, 8 and 10.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

4. Straight Lines and Bends | How LLMs Work: Foundations F04

LinesAndBendsVideo.mp4

Linear functions, and why networks need a bend.

This video has one big idea: linear layers alone are not enough. A matrix can stretch and rotate, but a stack of matrices is always just one more matrix. Putting a simple bend between the layers, a nonlinear function like ReLU or GELU, is what lets a neural network build curved, complicated functions.

Chapters

0:00 Linear: double in, double out
0:28 Linear layers collapse into one
0:38 What a straight map can't do
0:51 The fix: a bend called ReLU
1:08 GELU: a smooth ReLU
1:20 Curves from many straight pieces · A layer in code

Foundations F04, a pre-series for How LLMs Work. Used in episodes 8 and 9.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

5. Exponentials and Logarithms | How LLMs Work: Foundations F05

ExpLogVideo.mp4

Fast growth, and the function that undoes it.

This video is about two functions that appear all over LLMs. The exponential e^x grows by multiplying: it is always positive, and it turns small differences into big ones. The logarithm $\ln x$ undoes it: it turns probabilities into negative numbers that are easy to compare, and it turns multiplication into addition.

Chapters

0:00 Growth by multiplying
0:26 The number e , and e^x is always positive
0:47 e^x exaggerates differences
1:00 The logarithm undoes the exponential
1:13 Logs of probabilities
1:29 Multiplication becomes addition
1:41 exp and log in NumPy

Foundations F05, a pre-series for How LLMs Work. Used in episodes 6, 10, 11 and 12.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

6. Probability and Sampling | How LLMs Work: Foundations F06

ProbabilityVideo.mp4

Distributions, weighted dice, and randomness.

This video is about what an LLM actually outputs: not a word, but a probability distribution over words. You will see what a probability is, why a distribution adds up to one, how sampling picks one word at random according to it (a weighted spinner), and how a seed makes that randomness repeatable.

Chapters

0:00 A probability is a number from 0 to 1

0:26 A distribution adds up to one

0:43 Sampling: a weighted spinner

0:53 Many spins match the probabilities · Why sample? Variety

1:10 Seeds make randomness repeatable

1:23 Sampling in NumPy

Foundations F06, a pre-series for How LLMs Work. Used in episodes 1, 10 and 14.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

7. Softmax, Properly | How LLMs Work: Foundations F07

SoftmaxVideo.mp4

From any scores to probabilities.

This video has one big idea: softmax turns any list of scores into probabilities, in two steps. Exponentiate every score, then divide by the total. Everything else follows from those two steps: why it is “soft”, why only the differences between scores matter, how temperature sharpens or flattens the result, and why code subtracts the largest score first.

Chapters

0:00 Scores in, probabilities out

0:36 Working it through: 3, 2, 0

0:51 A soft version of the max

1:02 Only the differences matter · Temperature: sharper or flatter

1:20 Subtract the max: softmax in code

Foundations F07, a pre-series for How LLMs Work. Used in episodes 5, 6 and 10.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

8. Averages and Spread | How LLMs Work: Foundations F08

SpreadVideo.mp4

Mean, standard deviation, and keeping numbers tame.

This video has one big idea: two simple statistics describe a list of numbers, the mean (its center) and the standard deviation (its spread), and together they let us keep numbers tame. Normalizing with them is what layer norm does, and the same idea of spread explains why attention divides its scores by $\sqrt{d}$.

Chapters

0:00 The mean: the center

0:28 Standard deviation: the typical distance

0:52 Normalizing: mean 0, spread 1

1:14 The spread of a sum grows like $\sqrt{n}$

1:26 Dividing by $\sqrt{d}$ keeps softmax tame

Foundations F08, a pre-series for How LLMs Work. Used in episodes 6 and 9.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

9. Waves and Rotations | How LLMs Work: Foundations F09

WavesVideo.mp4

Sine, cosine, and turning arrows.

This video has one big idea: sine and cosine come from a point moving around a circle. Its height traces a wave that repeats every full turn, 2π , and the same two functions can turn an arrow by any angle without changing its length. Episode 4 uses both: waves of different frequencies to mark positions, and rotations that turn relative position into a relative angle.

Chapters

0:00 Sine and cosine on the unit circle

0:25 The sine wave, and a full turn of 2π

0:47 Frequency: how fast it repeats

0:58 Many waves: a fingerprint for every position · Rotating an arrow

1:18 Relative angles, and rotation in code

Foundations F09, a pre-series for How LLMs Work. Used in episode 4.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

10. Slopes and Gradients | How LLMs Work: Foundations F10

GradientsVideo.mp4

How a model knows which way to adjust.

This video answers one question: when a model is wrong, which way should each weight move? The answer is the slope: how fast the output changes when you nudge the input. Take a small step against the slope, repeat, and you have gradient descent, the way LLMs are trained.

Chapters

0:00 What a slope measures

0:26 The derivative: the slope at one point

0:41 Step against the slope: gradient descent

1:06 The gradient: one slope per weight

1:17 Slopes multiply: the chain rule and backprop

1:40 Gradient descent in code

Foundations F10, a pre-series for How LLMs Work. Used in episodes 11 and 12.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

11. NumPy in Three Minutes | How LLMs Work: Foundations F11

NumPyVideo.mp4

Just enough to read the code in the series.

This video is a quick tour of NumPy, Python's library for fast math on arrays of numbers. It covers just enough to read every code snippet in the series: shapes, element-wise arithmetic, the @ sign, axis and keepdims, broadcasting, reshape and masks.

Chapters

0:00 Arrays and shapes

0:29 Arithmetic on every element at once

0:40 The @ sign: matrix multiplication

0:50 Summing along an axis

1:02 Broadcasting

1:16 Reshape and transpose

1:26 Masks, and it all together

Foundations F11, a pre-series for How LLMs Work. The code in episodes 3 to 13 uses these NumPy basics.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

12. Neural Networks in Three Minutes | How LLMs Work: Foundations F12

NeuralNetVideo.mp4

Neurons, layers, weights, and learning.

This video has one big idea: a neural network is just a function, built from one very simple part repeated many times. A neuron weighs its inputs, adds a bias and bends the result; neurons side by side make a layer, and stacked layers make a network. Its numbers, the parameters, are not written by anyone: they are learned from data.

Chapters

0:00 A neuron: weigh, add, bend

0:42 A layer is one matrix multiplication

0:54 Stacking layers · Parameters: the numbers that learn

1:15 Inference and training

1:30 We choose the shape, data does the rest · Neurons and layers in code

Foundations F12 (extra), a pre-series for How LLMs Work. Used in episodes 8, 11 and 12.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

13. PyTorch and Autograd | How LLMs Work: Foundations F13

AutogradVideo.mp4

How one line computes every gradient.

This video opens up the one line that does all the calculus in training: `loss.backward()`. PyTorch records every operation on a tensor as a graph, then walks that graph in reverse with the chain rule. One call fills in the gradient of every weight, and an optimizer uses those gradients to update the weights.

Chapters

0:00 Tensors that remember

0:31 The forward pass builds a graph

0:45 `backward()`: the chain rule in reverse

1:04 One backward call, every gradient

1:15 The optimizer, and `zero_grad`

1:30 Autograd in code

Foundations F13 (extra), a pre-series for How LLMs Work. Used in episodes 11 and 12.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

14. Train vs Validation Data | How LLMs Work: Foundations F14

TrainValVideo.mp4

How we know a model really learned.

This video answers one question: how do we know a model has really learned, and not just memorized? Before training we split the data, never train on one slice, the validation set, and measure the loss there too. Comparing the two losses tells us whether the model is learning or overfitting.

Chapters

0:00 A low training loss can fool us

0:26 Split the data before training

0:50 Two losses, and the gap between them

1:13 Overfitting

1:25 The test set: one final, honest score · The split in code

Foundations F14 (extra), a pre-series for How LLMs Work. Used in episodes 11 and 12.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Training text: Tiny Shakespeare, from Andrej Karpathy's char-rnn project.

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

Playlist: How LLMs Work

A large language model, built up from scratch: tokens, embeddings, position, attention, the MLP, transformer blocks, probabilities, training, a real tiny GPT, and two bonus episodes on the KV cache and on turning a GPT into a chatbot. New to the math? Start with the How LLMs Work: Foundations playlist.

1. What is an LLM? | How LLMs Work, Ep. 1

WhatAnLLMDoes.mp4

How a language model writes, one word at a time.

This video has one big idea: a large language model does one simple thing, over and over. It predicts what comes next, picks something, adds it to the text, and repeats. Everything else in the series is about how the prediction works on the inside.

Chapters

0:00 One job: predict what comes next

0:30 A probability for every word

0:42 Pick one word and append it · The loop: predict, pick, append, repeat

1:03 The loop in code

1:16 Inside the model: a function made of numbers

Episode 1 of the How LLMs Work series. Background math in Foundations F01 (Vectors: Lists of Numbers as Arrows) and F06 (Probability and Sampling).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

2. Tokens: Chopping Text into Pieces | How LLMs Work, Ep. 2

TokensVideo.mp4

How text becomes numbers a model can read.

This video answers a question episode 1 skipped: what exactly goes into the model? A neural network only works with numbers, so text is first chopped into tokens, usually pieces of words, and each token is replaced by a number, its ID. The pieces are not chosen by hand: an algorithm called byte pair encoding learns them by counting which letters appear side by side.

Chapters

0:00 Text in, numbers out: the tokenizer
0:29 Two extremes: characters or words
0:58 Subword tokens: the middle road
1:16 Byte pair encoding: merge the most frequent pair
1:50 BPE training in code
2:01 Vocabulary and token IDs
2:22 The model never sees letters

Episode 2 of the How LLMs Work series.

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

3. Embeddings: Words as Vectors | How LLMs Work, Ep. 3

EmbeddingsVideo.mp4

How numbers start to carry meaning.

This video has one big idea: every token gets a vector, a list of numbers that places it as a point in space, and after training, where a token sits carries meaning. Similar tokens end up close together, closeness can be measured with the dot product, and even directions can mean something.

Chapters

0:00 IDs are labels, vectors are points
0:30 The embedding matrix: one row per token
0:58 Similar tokens end up close together
1:12 Measuring closeness: dot product and cosine
1:31 Directions carry meaning
1:45 Learned, not designed
1:56 Embeddings in code

Episode 3 of the How LLMs Work series. Background math in Foundations F01 (Vectors: Lists of Numbers as Arrows), F02 (The Dot Product) and F11 (NumPy in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

4. Where Am I? Position | How LLMs Work, Ep. 4

PositionVideo.mp4

How a model knows the order of words.

Embeddings tell the model what each token is, but not where it is: “The cat sat on the mat” and “The mat sat on the cat” get exactly the same vectors. This video is about putting the order into the vectors themselves: first two simple ideas, then the sine waves of the original transformer, and finally the rotations (RoPE) used by most modern models.

Chapters

0:00 Same tokens, different meaning
0:31 Idea 1: add the position number
0:43 Idea 2: learn a vector per position
0:58 Waves at many speeds, like a clock
1:29 Position is added to meaning
1:41 Sinusoidal encoding in code
1:54 RoPE: rotate instead of add

Episode 4 of the How LLMs Work series. Background math in Foundations F01 (Vectors: Lists of Numbers as Arrows) and F09 (Waves and Rotations).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

5. Attention I: Tokens Talking to Each Other | How LLMs Work, Ep. 5

AttentionIntroVideo.mp4

How words borrow meaning from their neighbours.

This video has one big idea: attention lets every token look at the other tokens and borrow meaning from the ones that matter. Each token asks a question (its query), compares it with what every token offers (its key), and mixes in the information (the values) of the best matches.

Chapters

0:00 Context changes meaning
0:43 Attention weights: a budget of 1
0:55 Query, key and value
1:21 Matching with the dot product
1:33 Mixing in the values
1:44 Only looking backwards
1:55 Learned, not hand-written

Episode 5 of the How LLMs Work series. Background math in Foundations F02 (The Dot Product), F03 (Matrices: Many Dot Products at Once) and F07 (Softmax, Properly).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

6. Attention II: The Math | How LLMs Work, Ep. 6

AttentionMathVideo.mp4

Queries, keys and values, one matrix at a time.

This video has one big idea: all of attention, for every token at once, is a handful of matrix multiplications. Project the tokens to queries, keys and values, score every query against every key, scale, mask the future, softmax each row, and mix the values: $\text{softmax}(Q K^T / \sqrt{d}) \cdot V$.

Chapters

0:00 From X to Q, K and V
0:42 Scores: Q times K transposed
0:59 Scaling by $\sqrt{d}$
1:09 The causal mask
1:20 Softmax, row by row
1:31 Mixing the values
1:43 The whole formula, in NumPy

Episode 6 of the How LLMs Work series. Background math in Foundations F02 (The Dot Product), F03 (Matrices: Many Dot Products at Once), F05 (Exponentials and Logarithms), F07 (Softmax, Properly), F08 (Averages and Spread) and F11 (NumPy in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

7. Multi-Head Attention | How LLMs Work, Ep. 7

MultiHeadVideo.mp4

Many questions, asked at the same time.

This video has one big idea: instead of one big attention, run several smaller ones side by side, called heads. Each head has its own query, key and value matrices, so each can learn to look for something different. Their results are glued back together, mixed by one more learned matrix, and added to the token's vector.

Chapters

0:00 Several heads, side by side
0:29 Sharing out the vector
0:40 Different heads, different patterns
1:10 Glue, mix and add
1:27 Many views for the price of one
1:37 Multi-head attention in code

Episode 7 of the How LLMs Work series. Background math in Foundations F03 (Matrices: Many Dot Products at Once) and F11 (NumPy in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

8. The MLP: Where Facts Live | How LLMs Work, Ep. 8

MLPVideo.mp4

The part of the model that thinks on its own.

Attention moves information between tokens. This video is about the other half of every layer: the MLP, which works on each token's vector by itself. It expands the vector, bends it with a simple nonlinear function, and projects it back down. It holds most of each layer's weights, and much of the model's factual knowledge seems to live there.

Chapters

0:00 One token at a time
0:21 Expand, bend, project back
0:44 Neurons as detectors
1:01 GELU: quiet unless the pattern shows up
1:14 Why the bend matters
1:26 Recalling facts
1:44 Where the weights are, in code

Episode 8 of the How LLMs Work series. Background math in Foundations F02 (The Dot Product), F03 (Matrices: Many Dot Products at Once), F04 (Straight Lines and Bends) and F12 (Neural Networks in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

9. The Transformer Block | How LLMs Work, Ep. 9

TransformerBlockVideo.mp4

Attention and MLP, wired together and stacked.

We have both halves of a layer: attention, where tokens share information, and the MLP, where each token processes it. This video wires them into a transformer block: each half adds its result to a stream of vectors, with a layer norm in front of it. Then the block is stacked, 12 times in GPT-2 small.

Chapters

0:00 The residual stream
0:27 Add, don't replace
0:50 Layer norm
1:05 Wiring the block · Stack it
1:35 The block in code

Episode 9 of the How LLMs Work series. Background math in Foundations F01 (Vectors: Lists of Numbers as Arrows), F04 (Straight Lines and Bends), F08 (Averages and Spread) and F11 (NumPy in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

10. From Vectors Back to Words | How LLMs Work, Ep. 10

UnembedVideo.mp4

Logits, softmax, and choosing the next token.

After the last block, every token is still a vector. This video turns the last one back into a word: the unembedding matrix gives one score (logit) per token, softmax turns the scores into probabilities, and a sampling rule (greedy, temperature, top-k, top-p) picks the next token. That closes the loop from episode 1.

Chapters

0:00 From the last vector to logits

0:40 Each logit is a dot product · Softmax: scores to probabilities

1:02 Greedy picking and temperature

1:26 Top-k and top-p

1:40 One sentence, many lessons

1:52 Choosing the next token, in code

Episode 10 of the How LLMs Work series. Background math in Foundations F02 (The Dot Product), F03 (Matrices: Many Dot Products at Once), F05 (Exponentials and Logarithms), F06 (Probability and Sampling) and F07 (Softmax, Properly).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

11. Training: Learning from Mistakes | How LLMs Work, Ep. 11

TrainingVideo.mp4

Loss, gradients, and millions of tiny nudges.

This video answers the question left open in episode 1: where do the model's numbers come from? A new model starts with random weights. Training shows it text, measures how wrong each next-token guess was with one number, the loss, and nudges every weight a tiny bit in the direction that makes the loss smaller. Then it does that again, millions of times.

Chapters

0:00 The text is its own answer key
0:31 Cross-entropy: scoring a guess
0:57 Gradient descent: feeling our way downhill
1:23 The learning rate · Backpropagation
1:49 The training loop in code
2:12 A real loss curve

Episode 11 of the How LLMs Work series. Background math in Foundations F05 (Exponentials and Logarithms), F10 (Slopes and Gradients), F12 (Neural Networks in Three Minutes), F13 (PyTorch and Autograd) and F14 (Train vs Validation Data).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Training text: Tiny Shakespeare, from Andrej Karpathy's char-rnn project.

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

12. Build a Tiny GPT | How LLMs Work, Ep. 12

TinyGPTVideo.mp4

Every piece, in about a hundred lines of code.

This video puts the whole series into one real program: `tiny_gpt.py`, about a hundred lines of PyTorch. It reads a million characters of Shakespeare, builds a GPT with 818,241 parameters from the pieces of episodes 2–10, trains it with the loop of episode 11 on an ordinary CPU, and watches it go from noise to something that looks like a play.

Chapters

0:00 Every character is a token

0:35 Embeddings: token plus position · Inside a block: attention and the MLP

1:03 Logits out, and 818,241 parameters

1:21 Training on an ordinary CPU

1:35 Watching it learn

Episode 12 of the How LLMs Work series. Background math in Foundations F05 (Exponentials and Logarithms), F10 (Slopes and Gradients), F12 (Neural Networks in Three Minutes), F13 (PyTorch and Autograd) and F14 (Train vs Validation Data).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Training text: Tiny Shakespeare, from Andrej Karpathy's char-rnn project.

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

13. Making It Fast: the KV Cache | How LLMs Work, Ep. 13 (bonus)

KVCacheVideo.mp4

Why generation doesn't start over for every token.

This bonus video is about speed. Generating text means one model run per new token, and done naively every run redoes the work for the whole text. The KV cache saves each token's keys and values, so every step only has to process one new token. The price is memory.

Chapters

0:00 The naive loop repeats work

0:21 Old tokens never change

0:34 The KV cache: one new row per step

1:04 The price: memory

1:20 Prefill and decode

1:30 Shrinking the cache: grouped-query attention

1:41 The KV cache in code

Episode 13 of the How LLMs Work series (bonus episode). Background math in Foundations F11 (NumPy in Three Minutes).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).

14. From GPT to Chatbot | How LLMs Work, Ep. 14 (bonus)

ChatbotVideo.mp4

How a text predictor learns to be an assistant.

This finale answers one question: how does a next-token predictor become a helpful assistant? It is trained in stages: pretraining on internet text, supervised fine-tuning on example conversations, then learning from preferences and other feedback. The machine never changes; only the data and the training signal do.

Chapters

0:00 Pretraining makes a base model

0:31 Supervised fine-tuning

0:47 A chat is one long document

1:00 Learning from preferences

1:11 RLHF and DPO

1:29 AI feedback and checkable answers

1:40 Same machine, different data

Episode 14 of the How LLMs Work series (bonus episode). Background math in Foundations F06 (Probability and Sampling).

Credits

Animated with Manim Community Edition (<https://www.manim.community>), the community-maintained version of Manim, the math animation engine created by Grant Sanderson (3Blue1Brown).

Made with AI assistance

The script, animation code and examples in this video were created with the help of AI (Claude, by Anthropic). The narration is an AI-generated voice (Kokoro text-to-speech).